

1) GET/

The screenshot shows the Apidog web interface. On the left sidebar, the 'APIs' section is active, showing a list of endpoints. The main panel displays a 'GET New HTTP Endpoint' configuration. The request is set to 'GET' with the URL 'http://127.0.0.1:8000/'. The 'Send' button is visible. Below the request configuration, the 'Body' tab is selected, showing a JSON response in 'Pretty' format:

```
1 {
2   "message": "Welcome to To-Do REST API",
3   "swagger": "http://127.0.0.1:8000/docs"
4 }
```

The status bar at the bottom indicates a successful response (200) with a response time of 11ms and a status of 'Passed'. A 'Need support?' dialog box is visible in the bottom left corner.

2) GET/todos

The screenshot shows the Apidog web interface. The request is set to 'GET' with the URL 'http://127.0.0.1:8000/todos'. The 'Send' button is visible. Below the request configuration, the 'Body' tab is selected, showing an empty JSON array in 'Pretty' format:

```
1 []
```

The status bar at the bottom indicates a successful response (200) with a response time of 56ms and a status of 'Success'. However, a 'Response Validation' error is shown on the right side of the interface:

- Response data differs from endpoint spec
- 1. \$ should be object

A 'Need support?' dialog box is visible in the bottom left corner.

3)POST/todos/{id}

The screenshot shows the Apidog interface for a new HTTP endpoint. The endpoint is a GET request to `http://127.0.0.1:8000/todos/{id}`. The request body is set to JSON and contains the following content:

```
1 {
2   "title": "1st task of todolist"
3 }
```

The response is a 200 status code with a JSON body. The response body is displayed in the 'Actual Request' tab:

```
1 {
2   "id": 1,
3   "title": "1st task of todolist",
4   "completed": false
5 }
```

A validation error is shown on the right side of the interface:

- Response Validation
- Response data differs from endpoint spec
- 1. \$ should be object

4)PUT/todos/{id}

The screenshot shows the Apidog interface for a new HTTP endpoint. The endpoint is a PUT request to `http://127.0.0.1:8000/todos/{id}`. The request body is set to JSON and contains the following content:

```
1 {
2   "title": "lab completed",
3   "completed": "yes completed"
4 }
```

The response is a 405 status code with a JSON body. The response body is displayed in the 'Actual Request' tab:

```
1 {
2   "detail": "Method Not Allowed"
3 }
```

A validation error is shown on the right side of the interface:

- Response Validation
- HTTP Status Code should be 200

5) DELETE/todos/1

The screenshot displays the Apidog web application interface. On the left sidebar, the 'APIs' section is active, showing a list of endpoints. The main workspace is titled 'DEL New HTTP Endpoint' and shows a DELETE request to the URL 'http://127.0.0.1:8000/todos/1'. The request body is a JSON object:

```
{  "title": "lab completed",  "completed": "yes completed"}
```

. The response is a 200 status code with a JSON body:

```
{  "message": "Task deleted successfully"}
```

. The interface includes tabs for 'Request', 'Design', 'Preview', and 'Settings'. The 'Request' tab is active, showing the request details. The 'Body' tab is also active, showing the response body. The status bar at the bottom indicates 'Success (200)' and 'Passed'.

➔> Now Below Screen shots of Postman above are using Swagger UI

1) GET/

The screenshot displays the Postman application interface. At the top, there's a navigation bar with 'Home', 'Workspaces', and 'Explore' options. A search bar is present, and a notification banner states: 'You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.' The left sidebar shows a 'History' panel with a list of recent requests, including several GET and POST requests to 'http://127.0.0.1:8000'. The main workspace is currently editing a GET request to 'http://127.0.0.1:8000'. Below the URL bar, there are tabs for 'Params', 'Authorization', 'Headers (6)', 'Body', 'Pre-request Script', 'Tests', and 'Settings'. The 'Query Params' section is visible, showing a table with 'Key' and 'Value' columns. The 'Body' tab is selected, showing a JSON response in 'Pretty' format:

```
{  "message": "Welcome to To-Do REST API",  "swagger": "http://127.0.0.1:8000/docs"}
```

 The status bar at the bottom indicates 'Status: 200 OK', 'Time: 8 ms', and 'Size: 203 B'. The Windows taskbar is visible at the very bottom, showing the time as 8:41 am on 09/07/2026.

2) GET/todos

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import POST http://127.0.0.1:5000/todos GET http://127.0.0.1:8000/todos

Today

- GET http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/json#39;
- GET http://127.0.0.1:5000/json#39;
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- GET http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

http://127.0.0.1:8000/todos

GET http://127.0.0.1:8000/todos

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (4) Test Results Status: 200 OK Time: 7 ms Size: 229 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 2,
3   "title": "lab completed",
4   "completed": false
5 },
6 {
7   "id": 3,
8   "title": "lab completed",
9   "completed": false
10 }
11
```

Console Not connected to a Postman account

Type here to search

33°C Clear 8:43 am 09/07/2026

3) POST/todos

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import POST http://127.0.0.1:5000/todos POST http://127.0.0.1:8000/todos

Today

- POST http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/json#39;
- GET http://127.0.0.1:5000/json#39;
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- GET http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

http://127.0.0.1:8000/todos

POST http://127.0.0.1:8000/todos

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (4) Test Results Status: 422 Unprocessable Entity Time: 15 ms Size: 225 B Save Response

Pretty Raw Preview Visualize JSON

```
2 "detail": [
3   {
4     "type": "missing",
5     "loc": [
6       "body"
7     ],
8     "msg": "Field required",
9     "input": null
10   }
11 ]
12
```

Console Not connected to a Postman account

Type here to search

33°C Clear 8:44 am 09/07/2026

4) GET/todos/{id}

The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8000/todos/1`. The request is configured with the method `GET` and the URL `http://127.0.0.1:8000/todos/1`. The response status is `404 Not Found` with a time of `10 ms` and a size of `159 B`. The response body is `{\"detail\": \"Task not found\"}`.

History: Today

- GET http://127.0.0.1:8000/todos/1
- GET http://127.0.0.1:8000/todos/{id}
- POST http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/json'
- GET http://127.0.0.1:5000/json'
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

Console: Not connected to a Postman account

Type here to search

33°C Clear 8:45 am 09/07/2026

5) PUT/todos/{id}

The screenshot shows the Postman interface with a PUT request to `http://127.0.0.1:8000/todos/0`. The request is configured with the method `PUT` and the URL `http://127.0.0.1:8000/todos/0`. The response status is `422 Unprocessable Entity` with a time of `7 ms` and a size of `225 B`. The response body is `{\"detail\": [{\"type\": \"missing\", \"loc\": [\"body\"], \"msg\": \"Field required\", \"input\": null}]}`.

History: Today

- PUT http://127.0.0.1:8000/todos/0
- PUT http://127.0.0.1:8000/todos/{id}
- PUT http://127.0.0.1:8000/todos/1
- GET http://127.0.0.1:8000/todos/1
- GET http://127.0.0.1:8000/todos/{id}
- POST http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000/todos
- GET http://127.0.0.1:8000
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/todos
- POST http://127.0.0.1:5000/json'
- GET http://127.0.0.1:5000/json'

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

Console: Not connected to a Postman account

Type here to search

33°C Clear 8:47 am 09/07/2026

6) DELETE/todos/1

The screenshot displays the Postman API client interface. At the top, there's a navigation bar with 'Home', 'Workspaces', and 'Explore' options, along with a search bar and user account links. A notification banner states: 'You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.'

The left sidebar shows a 'History' panel with a list of recent requests. The main workspace is configured for a DELETE request to the endpoint `http://127.0.0.1:8000/todos/1`. The request is set to the 'DELETE' method. Below the URL bar, tabs for 'Params', 'Authorization', 'Headers (6)', 'Body', 'Pre-request Script', 'Tests', and 'Settings' are visible. The 'Query Params' section is currently empty.

The response section shows a status of **404 Not Found** with a response time of 8 ms and a size of 159 B. The response body is displayed in 'Pretty' format as a JSON object: `{\"detail\": \"Task not found\"}`.

At the bottom of the interface, a Windows taskbar is visible, showing the system clock at 8:48 am on 09/07/2025 and the weather as 33°C Clear.